

Retry Pattern in C# using Asynchronous Programming with Examples

In this article, I will discuss **Retry Pattern in C#** using Asynchronous Programming with Examples. Please read our previous article discussing **How to Create a Synchronous Method using Tasks in C#** with Examples.

Retry Pattern in C# using Asynchronous Programming

One application of Asynchronous Programming is to perform a Retry Pattern. The idea is that sometimes there will be operations we want to retry several times. However, we do not want to retry immediately, but we want to retry after a certain amount of time. For example, if we request an HTTP to a Web server, sometimes those operations fail, and we may not want to tell the user that there was an error immediately. We may want to retry the operation just in case the operation works this time.

Structure of Retry Pattern in C#:

The following image shows the basic structure of the Retry Pattern in C# Asynchronous Programming.

```
//It tells the number of time we will retry the operation if it is failing
//Off Course, if it is not felling then we will not retry
var RetryTimes = 3;

//The idea is that we don't want to immediately retry, but
//we may want to retry after a certain amount of time.
//In our case, it is five hundred milliseconds or half a second.
var WaitTime = 500;

//Based on the RetryTime and status of the Operation, it will execute the loop
//Atleast 1 time and maximum RetryTime
for (int i = 0; i < RetryTimes; i++)
{
    try
    {
        //Do the Operation
        //If the Operation Sucessful break the loop
        break;
    }
    catch (Exception Ex)
    {
        //If the operations throws an error
        //Log the Exception if you want

        //Wait for 500 milliseconds
        await Task.Delay(WaitTime);
    }
}
```

Here, the variable **RetryTimes** tells the number of times we will retry the operation if it fails. If it is not failing, then we will not retry. And we have set the value to 3, meaning it will retry the operation a maximum of 3 times.

And one more thing, we don't want to retry the operation immediately. We may want to retry the operation after a certain amount of time. Here, the parameter **WaitTime** specifies the time duration for retrying the operation. We have set the value of WaitTime to 500 milliseconds so that it will retry the operation after 500 milliseconds or half a second.

Then, we created the for loop using the try-catch block. This for loop will execute a minimum of 1 time and a maximum of 3 times as we set the RetryTimes value as 3.

Then, inside the try block, we will call our async operation. The operation may be an API call or an Async method call. If the operation is successful, we will break the loop and come out from the for loop. If the operation is unsuccessful, meaning if we get any exception from the API or the Async method (whatever the operation), then the catch block will handle that exception and execute the catch block. If you want, you can log the exception details and wait for 500 milliseconds before continuing the next iteration of the loop.

Example to understand Retry Pattern in C#:

The following example shows the Retry Pattern in C# using Asynchronous Programming.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            RetryMethod();

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        public static async void RetryMethod()
        {
            //It tells the number of times we will retry the operation if it fails
            //Of course, if it is not failing then we will not retry
            var RetryTimes = 3;
            //The idea is that we don't want to immediately retry, but
            //we may want to retry after a certain amount of time.
            //In our case, it is five hundred milliseconds or half a second.
            var WaitTime = 500;
            for (int i = 0; i < RetryTimes; i++)
            {
                try
                {
                    //Do the Operation
                    //If the Operation Successful break the loop
                }
            }
        }
    }
}
```

```

        await RetryOperation();
        Console.WriteLine("Operation Successful");
        break;
    }
    catch (Exception Ex)
    {
        //If the operations throws an error
        //Log the Exception if you want
        Console.WriteLine($"Retry {i+1}: Getting Exception : {Ex.Message}");
        //Wait for 500 milliseconds
        await Task.Delay(WaitTime);
    }
}

}

}

public static async Task RetryOperation()
{
    //Doing Some Processing
    await Task.Delay(500);
    //Throwing Exception so that retry will work
    throw new Exception("Exception Occurred in while Processing...");
}

}

}

```

Output:

```

Main Method Started
Main Method Completed
Retry 1: Getting Exception : Exception Occurred in while Processing...
Retry 2: Getting Exception : Exception Occurred in while Processing...
Retry 3: Getting Exception : Exception Occurred in while Processing...

```

Generic Retry Pattern in C# Asynchronous Programming:

In the previous example, we have seen how to create a Retry Pattern in Asynchronous Programming. If we want to apply the Retry Pattern at multiple places, we need to make the retry pattern generic. Let us see how we can do this. Please have a look at the following image.

```

//Func is a generate delagate which returns something, in our case it is returning a Task
//We are setting the default value for RetryTimes = 3 and WaitTime = 500 milliseconds
public static async Task Retry(Func<Task> fun, int RetryTimes = 3, int WaitTime = 500)
{
    for (int i = 0; i < RetryTimes; i++)
    {
        try
        {
            //Do the Operation
            //We are going to invoke whatever function the generic func delegate points to
            await fun();
            Console.WriteLine("Operation Successful");
            break;
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception : {Ex.Message}");
            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }
}

```

The above retry pattern does exactly the same thing as the previous one. The only difference is that this retry pattern can be used with multiple methods. Let us see an example for understanding. Please have a look at the below example. In the below example, using the generic retry pattern, we are invoking the RetryOperation1 and RetryOperation2 async methods.

```

using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            RetryMethod();

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        public static async void RetryMethod()
        {
            //It will retry 3 times, here the function is RetryOperation1
            await Retry(RetryOperation1);
            //It will retry 4 times, here the function is RetryOperation2
            await Retry(RetryOperation2,4);
        }
        //Generic Retry Method
    }
}

```

```

//Func is a generate delegate which returns something, in our case it
returning a Task
//We are setting the default value for RetryTimes = 3 and WaitTime = 5
milliseconds
public static async Task Retry(Func<Task> fun, int RetryTimes = 3, int
500)
{
    for (int i = 0; i < RetryTimes; i++)
    {
        try
        {
            //Do the Operation
            //We are going to invoke whatever function the generic fun
points to
            await fun();
            Console.WriteLine("Operation Successful");
            break;
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception :
{Ex.Message}");

            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }
}

public static async Task RetryOperation1()
{
    //Doing Some Processing
    await Task.Delay(500);
    //Throwing Exception so that retry will work
    throw new Exception("Exception Occurred in RetryOperation1");
}

public static async Task RetryOperation2()
{
    //Doing Some Processing
    await Task.Delay(500);
    //Throwing Exception so that retry will work
    throw new Exception("Exception Occurred in RetryOperation2");
}
}
}

```

Output:

```
Main Method Started
Main Method Completed
Retry 1: Getting Exception : Exception Occurred in RetryOperation1
Retry 2: Getting Exception : Exception Occurred in RetryOperation1
Retry 3: Getting Exception : Exception Occurred in RetryOperation1
Retry 1: Getting Exception : Exception Occurred in RetryOperation2
Retry 2: Getting Exception : Exception Occurred in RetryOperation2
Retry 3: Getting Exception : Exception Occurred in RetryOperation2
Retry 4: Getting Exception : Exception Occurred in RetryOperation2
```

Problems with the above Generic Retry Pattern?

In the above generic retry pattern, we have one problem. We are calling the Retry from the RetryMethod as follows:

```
await Retry(RetryOperation1);
```

What if I want to do something if the operation fails three times? Because of how we implemented the generic Retry method, it just continues executing without telling us that the operation was successful or there was an error. Let us modify the Retry method as follows. Here, we are reducing the for loop execution for 1 time to execute the operation last time outside the for loop, which will work.

```
//Func is a generate delagate which returns something, in our case it is returning a Task
//We are setting the default value for RetryTimes = 3 and WaitTime = 500 milliseconds
public static async Task Retry(Func<Task> fun, int RetryTimes = 3, int WaitTime = 500)
{
    //Reducing the for loop Exection for 1 time
    for (int i = 0; i < RetryTimes - 1; i++)
    {
        try
        {
            //Do the Operation
            //We are going to invoke whatever function the generic func delegate points to
            await fun();
            Console.WriteLine("Operation Successful");
            break;
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception : {Ex.Message}");
            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }

    //Final try to execute the operation
    await fun();
}
```

In the above code, we will get the RetryTimes value as 3, and then the loop will execute 2 times if the operation is unsuccessful. The last time will execute outside the for loop, and we are not handling the exception here, so it will throw an exception that will tell the operation was successful. Now, you can catch the exception from where you called the Retry method as follows:

```
try
{
    await Retry(RetryOperation1);
}
catch(Exception ex)
{
    Console.WriteLine("The Operation was Failed");
}
```

The complete example code is given below.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            RetryMethod();

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        public static async void RetryMethod()
        {
            //It will retry 3 times, here the function is RetryOperation1
            try
            {
                await Retry(RetryOperation1);
            }
            catch(Exception ex)
            {
                Console.WriteLine("The Operation was Failed");
            }
        }
        //Generic Retry Method
        //Func is a generate delegate which returns something, in our case it
        //returning a Task
        //We are setting the default value for RetryTimes = 3 and WaitTime = 5
        milliseconds
    }
}
```

```

500) public static async Task Retry(Func<Task> fun, int RetryTimes = 3, int
{
    //Reducing the for loop Exection for 1 time
    for (int i = 0; i < RetryTimes - 1; i++)
    {
        try
        {
            //Do the Operation
            //We are going to invoke whatever function the generic fun
points to
            await fun();
            Console.WriteLine("Operation Successful");
            break;
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception :
{Ex.Message}");
            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }
    //Final try to execute the operation
    await fun();
}
public static async Task RetryOperation1()
{
    //Doing Some Processing
    await Task.Delay(500);
    //Throwing Exception so that retry will work
    throw new Exception("Exception Occurred in RetryOperation1");
}
}
}

```

When you run the above code, you will get the following output. Here, you can see that we get two errors because the loop executes two times, and finally, we get that operation failed. This is because the final execution of the function is executed outside of the try-catch block.

```

Main Method Started
Main Method Completed
Retry 1: Getting Exception : Exception Occurred in RetryOperation1
Retry 2: Getting Exception : Exception Occurred in RetryOperation1
The Operation was Failed

```

Generic Async Retry Method with Returning Value in C#:

As of now, the way we implement the Generic Retry method is not returning any value. Now, let us create a generic Retry method to return a value. If you want to return a value, you need to use Task<T>. For a better understanding, please have a look at the below image. In the below, T represents the type of value that the operation is going to return.

```
//Func is a generate delagate which returns something, in our case it is returning a Task
//We are setting the default value for RetryTimes = 3 and WaitTime = 500 milliseconds
public static async Task<T> Retry<T>(Func<Task<T>> fun, int RetryTimes = 3, int WaitTime = 500)
{
    //Reducing the for loop Exection for 1 time
    for (int i = 0; i < RetryTimes - 1; i++)
    {
        try
        {
            //It will invoke whatever function the generic func delegate points to
            //We will return from here if the operation was successful
            return await fun();
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception : {Ex.Message}");
            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }

    //Final try to execute the operation
    return await fun();
}
```

To test the above Retry method, please create the following async method, which returns a string.

```
public static async Task<string> RetryOperationValueReturning()
{
    //Doing Some Processing and return the value
    await Task.Delay(500);
    //Throwing Exception so that retry will work
    throw new Exception("Exception Occurred in RetryOperation1");
}
```

The Complete Example code is given below.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

    Console.WriteLine("Main Method Started");
    RetryMethod();

    Console.WriteLine("Main Method Completed");
    Console.ReadKey();
}
public static async void RetryMethod()
{
    //It will retry 3 times, here the function is RetryOperation1
    try
    {
        var result = await Retry(RetryOperationValueReturning);
        Console.WriteLine(result);
    }
    catch(Exception ex)
    {
        Console.WriteLine("The Operation was Failed");
    }
}

//Generic Retry Method Returning Value
//Func is a generate delegate which returns something, in our case it
returning a Task
//We are setting the default value for RetryTimes = 3 and WaitTime = 5
milliseconds
public static async Task<T> Retry<T>(Func<Task<T>> fun, int RetryTimes,
WaitTime = 500)
{
    //Reducing the for loop Exection for 1 time
    for (int i = 0; i < RetryTimes - 1; i++)
    {
        try
        {
            //Do the Operation
            //We are going to invoke whatever function the generic fun
points to

            //We will return from here if the operation was successful
            return await fun();
        }
        catch (Exception Ex)
        {
            //If the operations throws an error
            //Log the Exception if you want
            Console.WriteLine($"Retry {i + 1}: Getting Exception :
{Ex.Message}");

            //Wait for 500 milliseconds
            await Task.Delay(WaitTime);
        }
    }
    //Final try to execute the operation

```

```

        return await fun();
    }
    public static async Task<string> RetryOperationValueReturning()
    {
        //Doing Some Processing and return the value
        await Task.Delay(500);
        //Uncomment the below code to successfully return a string
        //return "Operation Successful";
        //Throwing Exception so that retry will work
        throw new Exception("Exception Occurred in RetryOperation1");
    }
}
}

```

Output:

```

Main Method Started
Main Method Completed
Retry 1: Getting Exception : Exception Occurred in RetryOperation1
Retry 2: Getting Exception : Exception Occurred in RetryOperation1
The Operation was Failed

```

The following code will be successfully run.

```

using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            RetryMethod();

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        public static async void RetryMethod()
        {
            try
            {
                var result = await Retry(RetryOperationValueReturning);
                Console.WriteLine(result);
            }
            catch (Exception ex)
            {
                Console.WriteLine("The Operation was Failed");
            }
        }
    }
}

```

```

    }
    public static async Task<T> Retry<T>(Func<Task<T>> fun, int RetryTimes,
WaitTime = 500)
    {
        for (int i = 0; i < RetryTimes - 1; i++)
        {
            try
            {
                return await fun();
            }
            catch (Exception Ex)
            {
                Console.WriteLine($"Retry {i + 1}: Getting Exception :
{Ex.Message}");
                await Task.Delay(WaitTime);
            }
        }
        return await fun();
    }
    public static async Task<string> RetryOperationValueReturning()
    {
        await Task.Delay(500);
        return "Operation Successful";
    }
}
}

```

Output:

```

Main Method Started
Main Method Completed
Operation Successful

```

So, we have implemented a Retry Pattern that allows us to centralize the logic of repeating an operation several times until it works or until we run out of retry trials.

When to Use Retry Pattern in C#?

The Retry Pattern is an important strategy in software development, useful in distributed systems, microservices architectures, or any scenario where a system interacts with external resources or services (like databases, APIs, or file systems). This pattern involves retrying a failed operation to handle transient faults that will likely disappear shortly. Here are scenarios where using the Retry Pattern in C# can be beneficial:

- **Handling Transient Failures:** The most common scenario for using the Retry Pattern is to handle transient errors. These errors occur sporadically and are often resolved quickly, such as temporary network glitches, brief service unavailability, or timeouts.
- **Interacting with External Services:** When your application communicates with external services (e.g., web services, cloud services), transient faults can occur due to the unpredictable nature of network communication. Retrying requests can often overcome these temporary issues.

- **Database Operations:** Database operations can sometimes fail due to transient issues like lock timeouts, network issues, or server overload. Implementing retries can make database interactions more robust.
- **Resilience in Microservices:** In a microservices architecture, services often depend on each other. The Retry Pattern can improve resilience by ensuring that a service can handle temporary failures in other services it depends on.
- **File I/O Operations:** File access can sometimes result in transient errors, especially in network file systems or cloud storage. Retries can handle these intermittent issues effectively.
- **Queueing Systems and Messaging:** When working with message queues or asynchronous messaging systems, transient failures like brief disconnections or high load can occur, making retries useful.

In the next article, I will discuss the [Only One Pattern in C#](#) Asynchronous Programming with Examples. In this article, I try to explain the **Retry Pattern in C#** Asynchronous Programming with Examples. I hope you enjoy this Retry Pattern in C# using Asynchronous Programming with Examples article.